

Traceability Data in Early Development Phases as an Enabler for Decision Support

Bashar Nassar
Chalmers University of Technology
Gothenburg, Sweden
bashar.h.nassar@gmail.com

Ali Shahrokni
Systemite AB
Gothenburg, Sweden
ali.shahrokni@systemite.se

Riccardo Scandariato
Chalmers and University of
Gothenburg
Gothenburg, Sweden
riccardo.scandariato@cse.gu.se

ABSTRACT

Traceability information between requirements, architectural elements and the results of test cases can be used to unearth interesting relationships between the early phases of the software development process and the software faults in the end product. For instance, complex dependencies between features and software components could lead to an increased level of flaws in the code. Such patterns can be detected and visualized as early warnings to the relevant stakeholders (e.g., the architect or the project manager). Ultimately, a fully-fledged prediction model can be developed if enough historical information is available from previous software projects. In this paper we introduce a method for building a decision support system based on historic product data.

Keywords

Traceability link; Requirement; Architecture; Test; Early development phases; Fault prediction.

1. INTRODUCTION

The problem tackled by this work is the early prediction of software defects, in order to reduce maintenance costs and improve quality. The context of this paper is the use of information extracted from requirements- and architecture-level information to predict test failures¹, which represent a reliable proxy for software faults. Previous work investigated the relation between the requirements and architecture, and how the requirements drive the architecture. For example, Nuseibeh [1] developed the Twin Peaks model to describe the co-development of requirements and architecture. Previous work has also investigated the relation between architecture and test, and how properties of the architecture can be used to predict faults. For instance, the work of Rathore and Gupta [5] shows that design attributes like coupling, complexity and size are correlated to fault proneness. However, the state-of-the-art is rather limited with respect to the whole chain of this relation, i.e., starting from requirements, to architecture, and ending with test.

In our work, we use data from large industrial projects and leverage the traceability data between requirements, architecture and tests in order to develop a fault prediction model, which could be used to generate early warnings to the software designers. In this paper, we outline our vision and present some initial results developed in an industrial setting.

The identification of faults can be done in different places throughout the development process. However, the later discovery of flaws leads to large fixing costs. In this work, the objective is to anticipate the faults at the early stages of the development process. The novel contribution of this work is the use of the relationships between requirements and high-level design (i.e., traceability links) to visualize sensitive points in the software system and possibly predict faults. An additional merit of this work is the use of empirical evidence. Realistic data about end-to-end relationships between the early artifacts and the discovered faults are not readily available to the research community. To overcome this limitation, we have partnered with a company specializing on managing traceability across artifacts in large projects. The uniqueness of the work comes from the tooling environment that stores the data traceability information. The tool enables programmed traversal of traceability links, which in turn enables continuous and dynamic data analysis in an actual industrial context

The paper is organized as follows. Section 2 discusses the related work. Section 3 presents the objectives of this research and how we can meet the objectives by using requirements and design metrics, together with traceability data in order to define a prediction model. Section 4 contextualizes our work and presents the concluding remarks.

2. RELATED WORK

Fault prediction has been intensively studied in academia and industry on different phases of development. A significant amount of the research has been done on the implementation level. Many researchers have tried to predict faults based on source code metrics such as size of code, code change history, source code complexity, complexity of the implementation processes, programming languages. One such study has investigated the effect of class size on fault-proneness [3]. Another study goes further to investigate the relation between the complexity of source code modification and the faults prediction [2].

Having early predictors (i.e., before the start of the testing or implementation phase) is useful and desirable as it can reduce the cost of software development, improve the quality, and increase the reliability of the system. In this respect, less research studies exist. For instance, some studies exist about predicting the system defects before the testing phase. A model using Object Oriented metrics was built to predict the faults during design phase. This model used

¹ In our work, we use the failures of test cases as a proxy for actual software faults.

neural network technique and Bayesian Regularization (BR) algorithm. The results of this study prove that the BR algorithm provides better accuracy than Levenberg-Marquardt (LM) and Back propagation (BPA) algorithms. In addition, the study finds that machine learning models are significantly used and provide superior results [6]. Moreover, other works exist about using design properties as predictors. For instance, Rathore and Gupta [5] investigated the relationship between the class design level object-oriented metrics with fault proneness of object-oriented software system. The results of their work showed that design attributes like coupling, complexity and size are correlated to fault proneness.

Over the last decade, researchers have studied on particular areas of the traceability problem and they tried to developing more complicated tooling, even they suggested many new research areas to be address [12]. Traceability has many uses it could be used to check if the requirements have been satisfied in advance development phase, like design and code. Also, it could help to assess and manage the impact of changing a system artifact, i.e., requirement, among many other artifacts[13]. Moreover, traceability has been used in fault based testing to generate test data to demonstrate the absence of a set of pre-specified faults [14].

3. DECISION SUPPORT FROM TRACEABILITY DATA

Our objective is to discover and highlight the critical relations between requirements and architectural design choices that could lead to software faults. To achieve our objective, we have analyzed three large projects from two companies. The raw data from these projects (primarily requirements documentation and architectural design) have been processed via an automated tool (which we built) in order to extract relevant metrics. In the investigated projects, the failure of the test cases is linked back to the appropriate requirements and design modules. Hence, the collected metrics also represent a starting point for the construction of a decision support system for the identification of software faults.

In the following sub-section, we briefly describe the raw data from the three projects, the collected metrics, the technical aspects of the metrics along with some challenges of extracting them in real-life projects, and the initial results for the decision support system.

3.1 Three Industrial Projects

In this work, we are using a data management platform called SystemWeaver, which is a holistic information management solution for systems engineering and software development [8]. The selection of this platform as a data model was based on its capability to manage the product development process on different level of abstraction, the full traceability between the product artifacts, and the support for testing. The raw data which we are using to investigate our research comes from two of the leading automotive companies in Sweden. They include both the software and hardware design for several mature and deployed products. The structure of the data is represented in Figure 1. The highest level of abstraction is the system level, which contains the system features and represents how the features interact with each other. For example, a vehicle includes the braking function, the cruise control function, etc. The second level is the analysis level, which contains an abstract description of the system architecture. The next level is the logical design, which is a more detailed description of the system design and can be viewed as a refinement of the level above. At the logical design level, the system design contains a set of connected components. Each component has a set of requirements,

functions, connection ports and some other attributes. The bottom-most level in the figure is the hardware architecture, which represents the hardware-level design of a vehicle as a network of electronic control units (ECUs).

The traceability information takes different forms. First, it includes the relations (called T.W in the figure) between the product artifacts at the same abstraction level, for instance, the traceability links between a software component, its requirements, and test case validating the component's behavior. Second, SystemWeaver also maintains the relations (T.L in the figure) between the product artifacts on different abstraction levels. Finally, the traceability information could include the relations (T.V in the figure) between product versions.

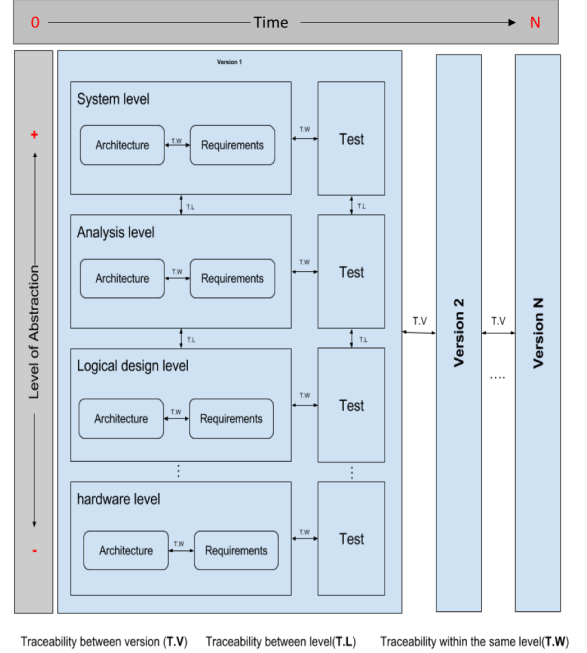


Figure 1. Traceability between system artifacts, abstraction level, and versioning.

3.2 Extraction of Useful Metrics

We have developed a module (called SmartTrace) that integrates with Systemite's SystemWeaver [9] and performs the automated extraction of several metrics out of the raw data mentioned in Section 3.1. This module calculates metrics that can be specified dynamically using path expressions, as discussed in [7] and Table 1 presents an example for some of the metrics that we had implemented in SmartTrace.

Metrics have been used intensively by the software community to measure the system attributes in different phase of the development process, in order to evaluate the quality of the system. As shown in the Table 1, the metrics that we extract for our decision support system include requirements, design, test, and traceability metrics.

The requirement metrics measure different aspects of the requirements like ambiguity, correctness, completeness, understandability, etc., [4]. The design metrics measure the design attributes, such as size, depth of inheritance, coupling, complexity, etc., [3]. The test metrics measure the testing attributes, i.e., number of test cases passed or failed, number of test cases under investigation and test case execution time. Finally, the traceability metrics measure the relations between different system artifacts,

those artifacts are requirement, component, and test cases. By applying more metrics, the fault prediction module will be more powerful, which will increase the probability of finding the fault patterns. Table 1 presents some of the metrics that we had implemented in SmartTrace fault prediction module, some of them are conventional metrics, where other are more innovative.

Table 1. Metrics suite used by our decision support system.

Category	Metric	Description
Requirement	Requirement Unambiguous	It indicates the degree of the requirement understandability.
	Requirement Completeness	It indicates the degree of the requirement completeness
Design	Cyclomatic Complexity	It indicates the complexity of a system, by counting the number of the independent paths through a system.
	Coupling	It measures the degree of interdependence between software components.
Test	Number of Test Case	It measures the total number of performed test cases for a specific part of a system, regardless of the result of the test case.
	Pass/Fail Test Case	It measures the number of passed/failed test case for a specific part of a system.
Traceability	Component Responsibility	It measures the number of requirements implemented by a component .
	Requirement Uses	It measures the number of components implementing a requirement.

3.3 Metrics implementation and challenges

We implemented the metrics using the path expression language in SystemWeaver, which navigates through the elements and attributes of the product. The result of the executed query is a set of model elements according to the XPath expression. The language supports the basic model operators, like div, and, or, and so on[10].

The following is a simple example for one of the implemented metric, i.e., the component interaction metric (CIM) [11]. This metric measures the component interaction degree, by calculating the ratio between the component's incoming (I) and outgoing (O) interaction. The formula of the metric is $CIM = I/O$.

This metric can be calculated using the following XPath expression, which includes the count function and div operator:

```
/ELEMENTS/COMPONENT-MODEL/IN-PORT.Count.Div(/ELEMENTS/COMPONENT-MODEL/OUT-PORT.Count)
```

Using Xpath to define metrics has several advantages. For example, it allows us to share metrics between different parties, who have the

same data model (meta model). This improves the communication between different tools or organization.

We had several challenges associated with extracting the metrics. For instance, the structure of the raw data could be different between products or companies, which required us to adapt our metrics definition to fit the studied data. Another challenge was to understand the data with very large size. Also, we faced some limitations in the XPath language, where not all types of metrics can be defined easily [7]. Last but not least, sometimes the raw data that we have is inconsistent or not complete, which required from us some manual activities to overcome the problem.

3.4 Towards a Decision Support System

The above-mentioned SmartTrace module is currently being extended into an information dashboard for decision support, which guides the engineers towards the identification of problematic components and requirements.

It is designed to mature continuously as the product development data in the organization grows. The data, often created by users, is used to generate insights on the patterns that are likely to result in failures. These insights are used to provide the user with decision support as shown in Figure 1.

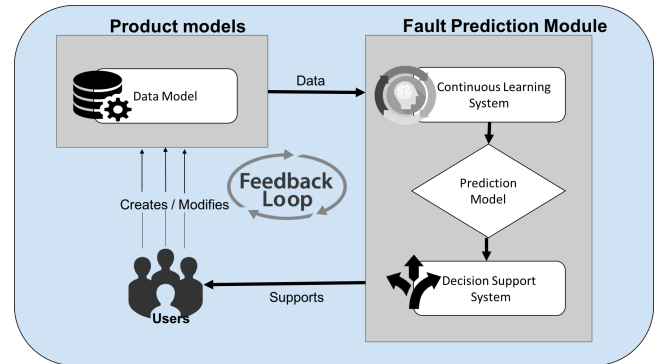


Figure 1. The SmartTrace fault prediction model using the traceability data in the early stage of development life cycle.

The users trigger the module by creating or modifying the product data. The traceability data between the product artifacts allows us to investigate the fault patterns, by applying statistical methods over the product data. We use the traceability between the product artifacts in addition to the isolated artifacts to retrieve these patterns. The first part of the module is a continuous learning system, which grows organically and continuously to update retrieved data from the product models, by adding the experience gained from new iteration to the old data. Over time the learning system will be more capable to provide the prediction module with necessary data to discover more potential faults.

Moving to the second part, which is the prediction model, this is responsible for predicting faults based on existing fault patterns, historical product data, and traceability data. The prediction module uses this data to find the faults or point to the places where the potential faults could be hidden. The last part of this module is the decision support system, which provides the users with real time decision support.

An essential part of this module deals with extracting the fault patterns. The patterns extraction process starts by defining and calculating metrics, these metrics measure a wide variety of product attributes, as mentioned before. The next step is to find the correlation between the calculated metrics and the related test

cases, we use standard statistical tools, like non-parametric tests (e.g., Wilcoxon). A significant correlation with test failures means that the metrics can be used to identify the product elements related to the metrics (e.g., an architectural component) as a source of faults. The decision support system recognizes these occurrences and check the product based on the available patterns. Accordingly, the decision support system warns the developers about the potential risk and provides them with suitable suggestions (e.g., how to refactor the system to improve on the relevant metrics). For instance, if the number of dependencies shows a likelihood of introducing failures in one component, the decision support can suggest to split the component into two smaller components or to restructure the allocation of software components onto the electronic control units (ECUs).

Currently, we are looking for an automated way to discover those correlations between metrics and test cases. Later, we will expand this work to investigate the use of several metrics at once as failures could be due to factors related to multiple metrics.

4. CONCLUSION

This paper has reported on our vision concerning the use of rich traceability data to build a decision support system able to generate early warnings in case of problematic requirements, ill-designed architectural components, critical deployments and so on. The feasibility of our approach has been shown by investigating three large industrial projects and most of the necessary building blocks have been already developed. At this point, the key part of the proposed solution (the SmartTrace module) is being finalized. In future work, we will validate SmartTrace on a larger set of projects, possibly from different application domains, as so far we have focused on the automotive industry only.

From a research perspective, the decision support system we are developing is expected to significantly contribute to the early detection of requirements- and design-level flaws. The objective is to increase the quality and reliability of software by reducing the number of software faults and fail cases. From an industrial perspective, the approach is meant to reduce the risk of a new project, by applying the historical and accumulated knowledge from the projects previously developed in a company. In addition, it helps the designers to be more open to accept the customer-requested changes (as issues can be debunked by the decision support system), which will positively affect the customer satisfaction.

5. REFERENCES

- [1] Nuseibeh, B., 2001. Weaving together requirements and architectures. *Computer*, 34(3), pp.115-119.
- [2] Hassan, A.E., 2009, May. Predicting faults using the complexity of code changes. In *Proceedings of the 31st International Conference on Software Engineering* (pp. 78-88). IEEE Computer Society.
- [3] Chen, J., Wang, H., Zhou, Y. and Bruda, S.D., 2011. Complexity metrics for component-based software systems. *International Journal of Digital Content Technology and its Applications*, 5(3), pp.235-244.
- [4] Bokhari, M.U. and Siddiqui, S.T., 2011, March. Metrics for requirements engineering and automated requirements tools. In *Proceedings of the 5th National Conference*.
- [5] Rathore, S.S. and Gupta, A., 2012, September. Investigating object-oriented design metrics to predict fault-proneness of software modules. In *Software Engineering (CONSEG), 2012 CSI Sixth International Conference on* (pp. 1-10). IEEE.
- [6] Mahajan, R., Gupta, S.K. and Bedi, R.K., 2015. Design of Software Fault Prediction Model Using BR Technique. *Procedia Computer Science*, 46, pp.849-858.
- [7] Söderberg, J., Shahrokni, A. and Nassar, B., 2016. Using XPath to Define Design Metrics. *International Conference on Performance, Safety and Robustness in Complex Systems and Applications (PESARO)*. IARIA, 2016.
- [8] Systemweaver.se, (2016). SystemWeaver - Managing product development information:: Home. [online] Available at: <http://www.systemweaver.se> [Accessed 14 Feb. 2016].
- [9] Systemite.se, (2016). Systemite AB | Keeping complex systems consistent, correct and complete.. [online] Available at: <http://www.systemite.se> [Accessed 15 Feb. 2016].
- [10] W3schools.com, (2016). XPath Tutorial. [online] Available at: http://www.w3schools.com/xsl/xpath_intro.asp [Accessed 16 Feb. 2016].
- [11] Chen, J., Yeap, W.K. and Bruda, S.D., 2009, May. A review of component coupling metrics for component-based development. In *World Congress on Software Engineering* (pp. 65-69). IEEE.
- [12] Cleland-Huang, J., Gotel, O.C., Huffman Hayes, J., Mäder, P. and Zisman, A., 2014, May. Software traceability: trends and future directions. In *Proceedings of the on Future of Software Engineering* (pp. 55-69). ACM.
- [13] Cleland-Huang, J., Gotel, O. and Zisman, A., 2012. *Software and systems traceability* (Vol. 2, No. 3, pp. 7-8). Springer.
- [14] Hayes, J.H., 2003, November. Building a requirement fault taxonomy: Experiences from a NASA verification and validation research project. In *Software Reliability Engineering, 2003. ISSRE 2003. 14th International Symposium on* (pp. 49-59). IEEE.